第1章 引论

黄建军
中国人民大学信息学院

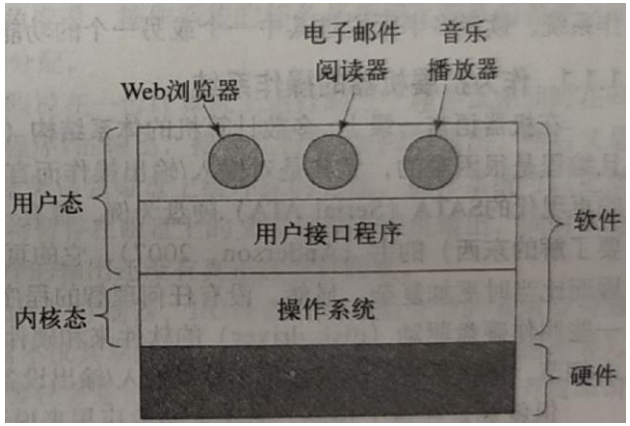
目录

- 1 操作系统概念
- 2 Xinu方法
- 3 硬件与运行时环境概述
- 4 代码开发基础
- 5 Xinu运行效果
- 6 推荐阅读

1 操作系统概念

- 复杂的现代计算机系统：一个或多个处理器、主存、磁盘、打印机、键盘、鼠标、显示器、网络接口及各种其他输入输出设备
  - 管理这些部件并加以优化使用，是一件挑战性极强的工作
- 计算机安装了一层软件，称为操作系统（Operating System），其任务是为用户程序提供一个更好、更简单、更清晰的计算机模型，并管理上述所有设备
- 常见操作系统包括Windows、Linux、MacOS、iOS、Android、HarmonyOS等，用户与操作系统之间进行交互通常有两种方式
  - 基于文本进行交互的程序通常称为shell（Linux上的Bash、zsh等）
  - 基于图标进行交互的程序称为图形用户界面（GUI）
- 表面现象是会骗人的：这些交互程序并不是操作系统的一部分

基本层次结构



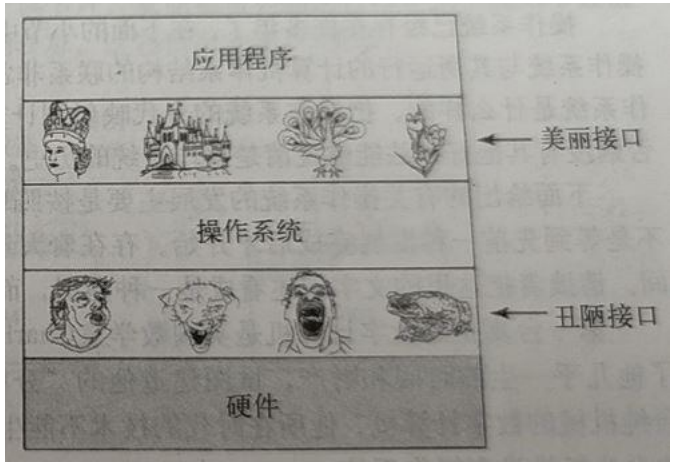
- 右图展示了一个简化视图
  - 底层是硬件：芯片、电路板、磁盘、键盘等
  - 硬件之上是软件
- 多数计算机有两种运行模式：**内核态**和**用户态**
  - 软件中最基础的操作系统运行在内核态，具有对所有硬件的完全访问权，可以执行机器能够运行的任何指令。
  - 软件其余部分运行在用户态，在该模式下，只是用了机器指令中的一个子集。特别地，那些会影响机器的控制或可进行I/O操作的指令，在用户态的程序里是禁止的（如x86上的in、out指令）。
- 用户接口程序（shell或GUI）一般处于用户态程序中的最低层次

大型的操作系统

- 一般的操作系统具有大型、复杂、长寿的特点
- 当代Linux和Windows操作系统的源代码都有上千万行
  - 数百本每本1000页、每页50行的书
- 操作系统编写完成后，会在长时间内进行演化
  - 如Windows操作系统系列、UNIX/Linux等

什么是操作系统

- 操作系统的两个基本任务：为应用程序提供一个资源集的清晰抽象，并管理硬件资源（而不仅仅是一堆硬件）
- 作为扩展机器的操作系统：
  - 在机器语言一级上进行编程是很困难的，尤其对输入输出操作而言
  - 以SATA硬盘为例，一本描述早期版本硬盘接口的书超过450页，自2007年起，又被修改过很多次
  - 有理智的程序员不会想要在硬件层面与硬盘打交道；相反，他们使用被称为硬盘驱动的软件来和硬件打交道，这些软件提供了读写硬盘块的接口
  - 即使在这个层面，对应用程序来说还是太底层了，因此操作系统又提供了又一层抽象：**文件**，使得程序读写文件不用处理硬件实际工作中的细节



什么是操作系统

- 作为资源管理者的操作系统：
  - 在相互竞争的程序之间有效地控制对处理器、存储器以及其他I/O接口设备的分配
  - 资源管理包括两种方式实现多路复用（共享）资源：在时间上复用和在空间上复用
- 在时间上复用：
  - 不同程序或用户轮流使用一种资源，如多个程序轮流在同一个CPU上运行
  - 操作系统控制如何实现时间复用——谁是下一个以及运行多长时间等
- 在空间上复用：
  - 每个客户都得到资源的一部分，从而取代了客户排队
  - 如：在若干程序之间分割内存，使得每个运行程序都可同时入驻内存
  - 操作系统则需要解决这种做法导致的公平、保护等问题

万物互联时代操作系统的关键特征

- 万物互联时代已来临
  - 2025年智能设备数量将达到300亿部，IoT硬件技术在计算单元、存储单元、通信单元等方面能力增强，在智能驾驶、智能家居等领域大行其道。
- 智能终端操作系统需要为新型硬件和较低性能非智能终端提供支持，与其他智能终端更高效地协同，交互更人性化、更智能，体验更好。
- 下一代操作系统的关键特征：
  - 设备种类异常丰富，开发者需要一种能够灵活定制的操作系统，完全按照模块化方法构建，具有高度抽象和模块化的特点，系统构建者可以从现有货架中快速组装；
  - 具备细粒度的跨设备虚拟化；
  - 提供多模态交互，提供更自然、更高效、多样化的交互方式，同时各种交互方式互相叠加，主动交互与被动交互协同，给人们带来前所未有的交互体验；
  - 具备异构分布式能力，分布式能力不再是服务器的专属能力；
  - 原生支持AI，包括AI基础能力（如：对AI硬件的管理、内置AI开发框架等）和利用AI自主改造的能力（如：通过周边环境感知、运行上下文感知等AI技术让系统资源的管理更加高效，通过语音识别、机器视觉等AI能力让交互操作个性化、更自然等）；
  - 更加安全，即通过信任管理、访问控制、差分隐私、数据脱敏等技术，构建一体化的数据安全体系。

内容来源：《鸿蒙操作系统设计原理与架构》

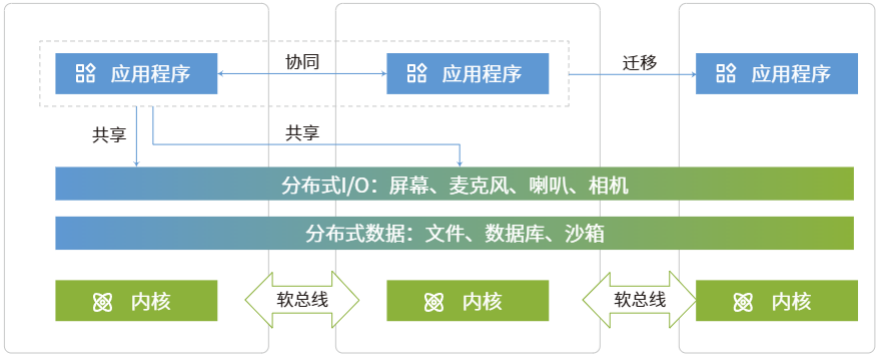
HarmonyOS的基本设计理念

- 在设备多样的场景下，突破单设备的能力局限，加速信息在设备之间顺畅地流动，为用户提供完整、一致和便捷的分布式体验。

超级终端

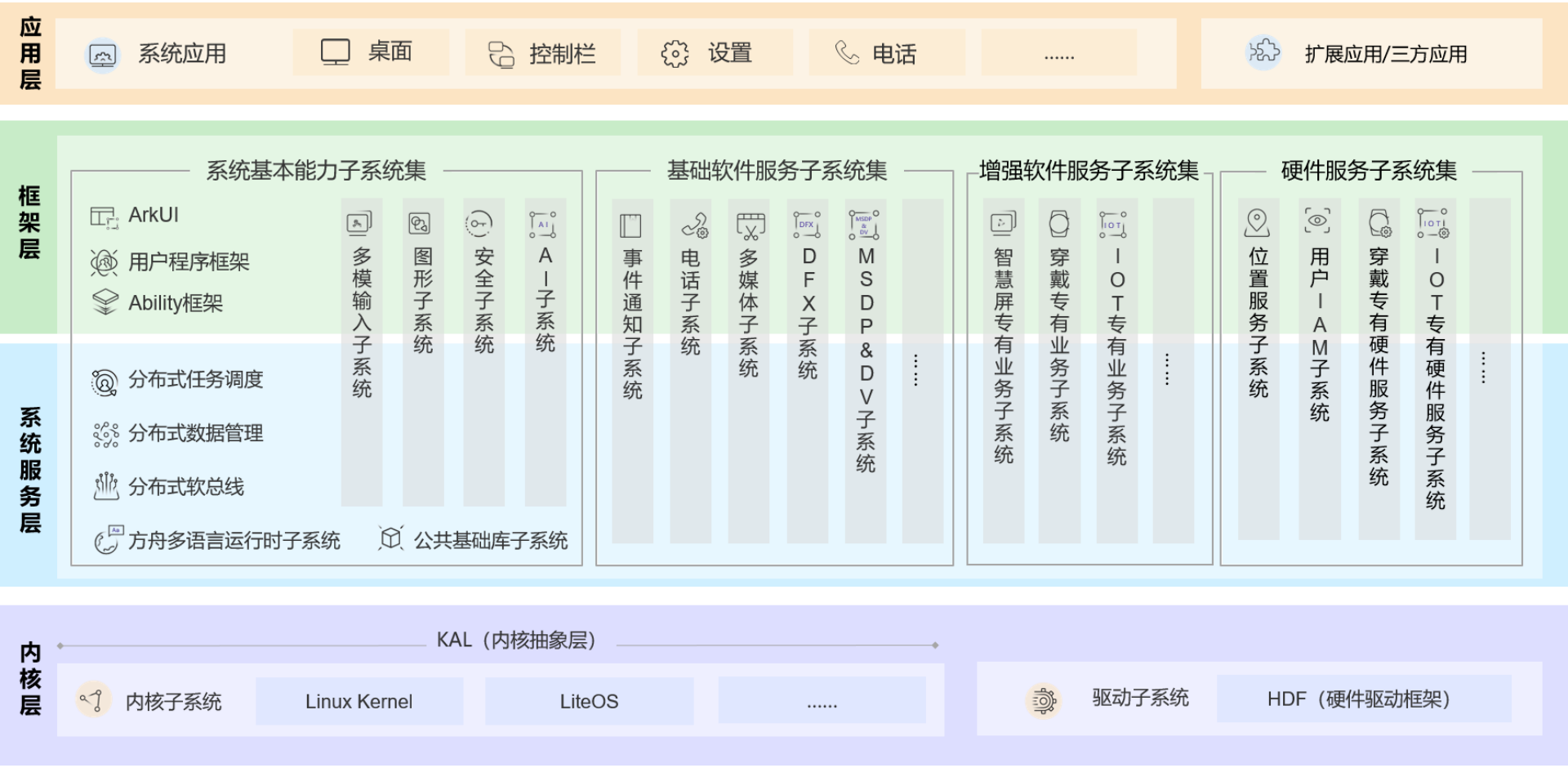
- 超级终端是指用户在不同场景下使用的各种智能终端通过HarmonyOS自动系统组成的逻辑终端。它包含各种类型的智能终端，是HarmonyOS管理的抽象终端，对用户而言，它就像一个终端。

多设备的硬件融为一个超级终端的技术架构



数据共享，硬件互助，分布式协同的技术特性

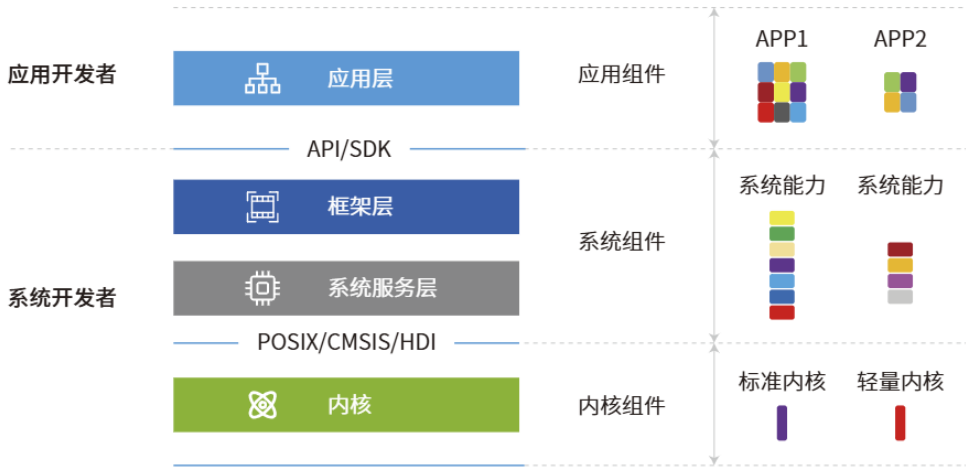
HarmonyOS的分层架构

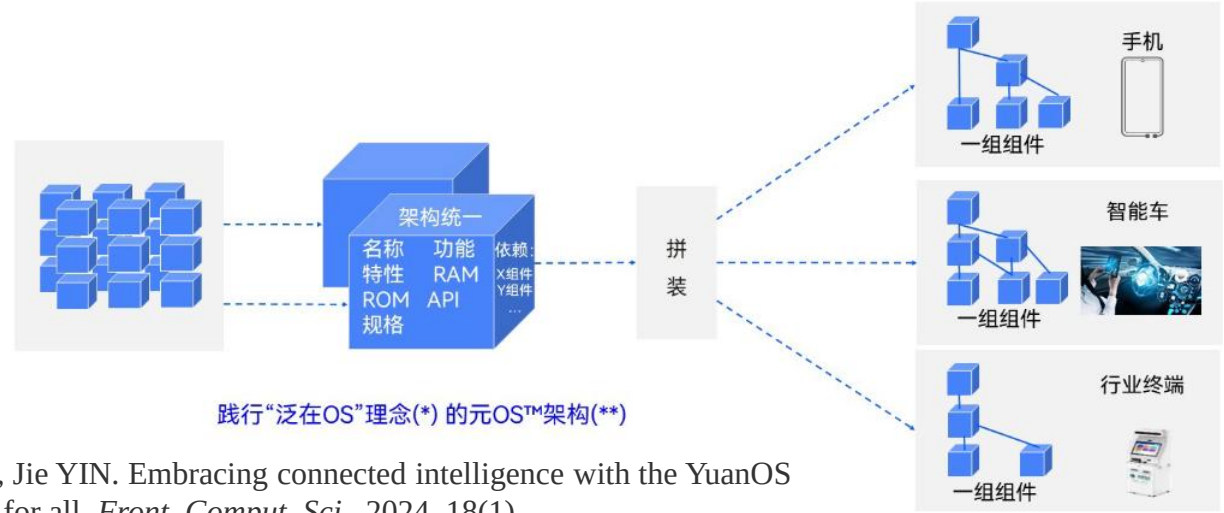


https://gitcode.com/openharmony

HarmonyOS的部件化架构设计

- 在架构分层的基础上，HarmonyOS设计了积木式的部件化架构，满足系统**架构解耦**，支持不同规格操作系统的**弹性部署**。





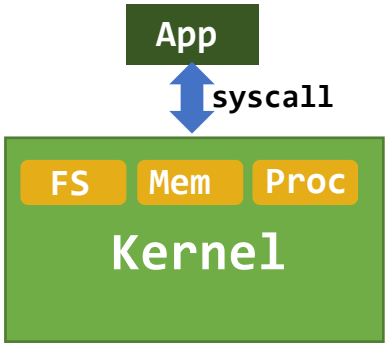
Haibo CHEN, Ning JIA, Jie YIN. Embracing connected intelligence with the YuanOS architecture: one OS kit for all. *Front. Comput. Sci.*, 2024, 18(1)

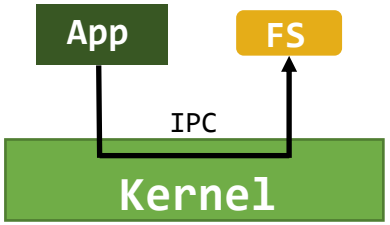
内核

- 供应商出售的大型商业操作系统通常包括很多额外的软件组件
  - 如：编译器、链接器、加载器、库函数和一系列应用程序
- 为了区分这些额外的软件和基本的操作系统，我们有时会用**内核**这个术语，指代常驻在内存中并且提供诸如并发进程支持等关键性服务的代码
  - 例：`Linux kernel v.s. Ubuntu Linux`
- 本课堂中，操作系统这个术语指的就是内核

宏（Monolithic）内核与微内核（Micro）

- 宏内核（Monolithic Kernel，单内核）在其内核中包含其所有（或至少绝大部分）服务，减少了所涉及的上下文切换和消息传递的数量。
  - Linux， MacOS <= 8.6， Most BSDs， xv6， MSDOS， Windows 9x
- 微内核（Micro Kernel）尝试将大多数服务（例如网络、文件系统等）作为用户空间中的守护程序/服务器运行，剩下的内核要做的就是基本服务，例如物理内存分配，调度和消息传递（进程间通信）。
  - 理论上，微内核更具响应性，许多功能驻留在用户空间线程和进程中，无需将上下文切换为内核，但涉及的消息传递和上下文切换的数量多，理论上比宏内核慢。
  - Mach，QNX，L4，Minix，LiteOS

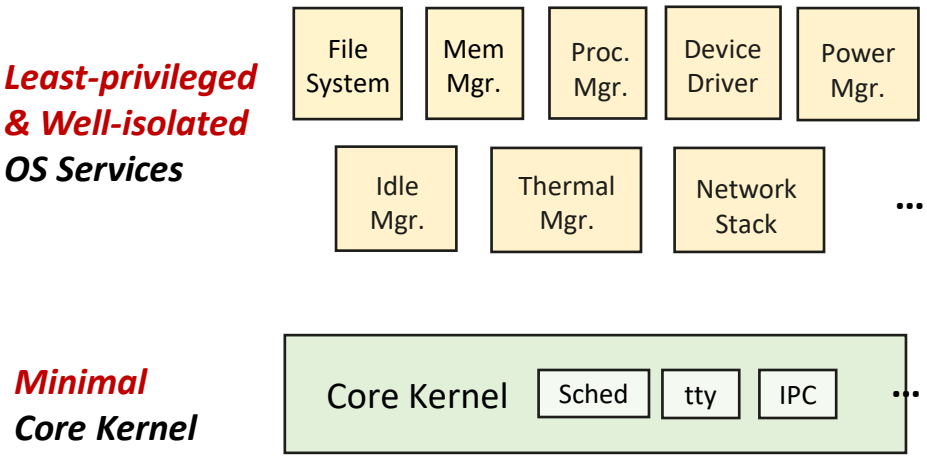




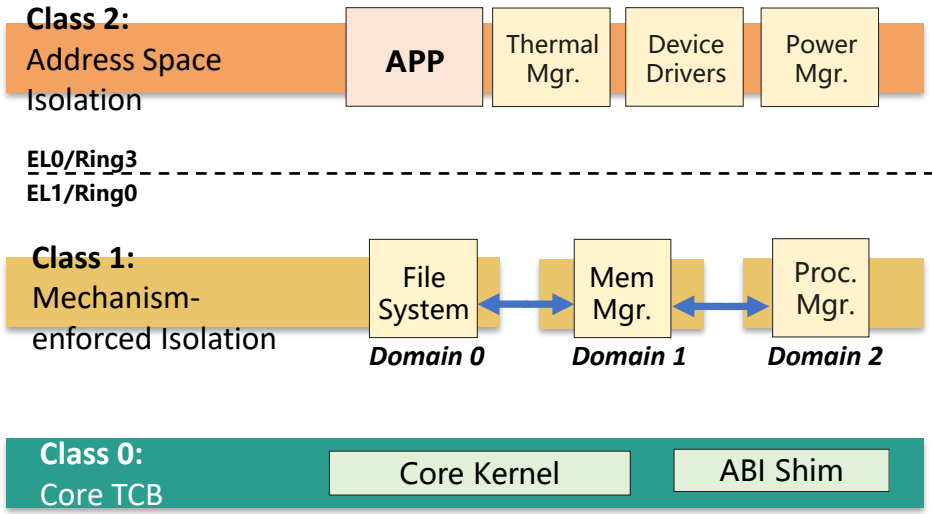
HUAWEI LiteOS

HarmonyOS的微内核设计

- 保留最小化核心内核设计
- 引入机制增强的隔离类来放宽隔离限制、加速IPC



HarmonyOS Kernel



Differentiated Isolation Classes** in **HarmonyOS

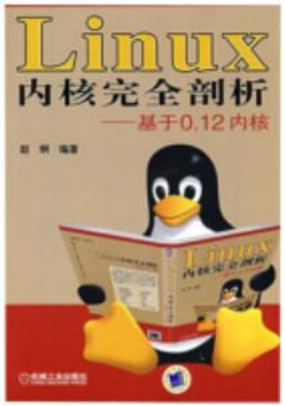
from Niu Liu's OSDI talk

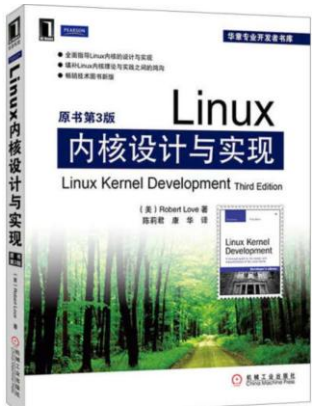
2 Xinu方法

- Xinu是普渡大学Douglas Comer创建的一个小而美的操作系统，最早可追溯到1979年
- 自1996年开始，Xinu逐渐在世界上很多知名大学的操作系统课程教学中引入
- 目前Xinu可以在MIPS、ARM、x86等平台上运行
  - 参考书目《操作系统设计：Xinu方法》（第2版）中同时介绍了在x86（Intel Galileo实验板）和ARM（BeagleBone Black实验板）平台上的实现细节
  - 本课堂主要采用QEMU的x86模拟器进行实验
- Xinu在商业产品中也得到了广泛应用
  - Mantissa Corporation将Xinu移植到IBM z9
  - 针对Linksys路由器开发的MIPS版XINU（参考书第一版）
  - 使用在Williams/Bally出售的弹球游戏中
  - Woodward公司使用Xinu系统来控制大型燃气/蒸汽和柴油/蒸汽涡轮发动机
  - ……

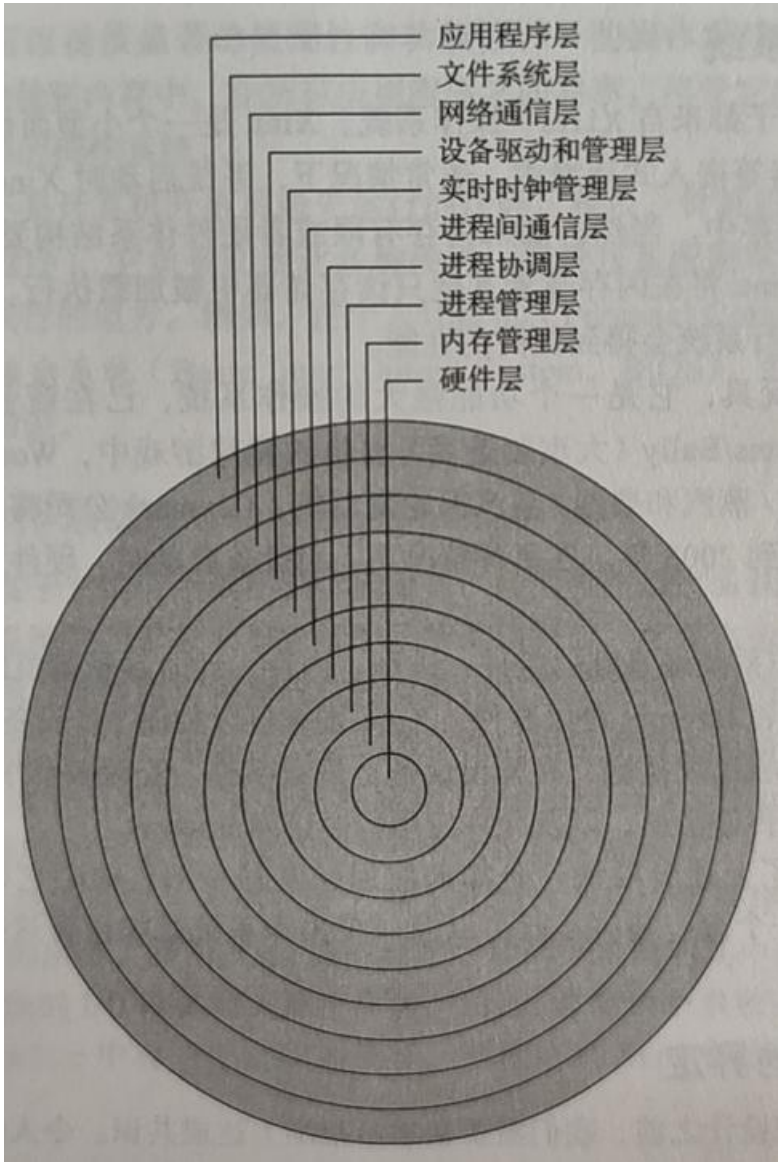
研究方法

- 将介绍如何构建、设计并且实现操作系统内核，使用了工程学方法
  - 不是仅仅调研现有操作系统、罗列它们的特性和抽象地对其进行描述
- 代码是课堂讲解的一个重要组成部分——事实上，代码是讨论的核心
  - 通过阅读和学习所罗列的程序来欣赏其中的微妙之处和工程中的细节
  - 样例代码非常精简，意味着可以集中精力在概念的理解上，而不需要费力地阅读许多页的代码
  - 更进一步理解现代操作系统的核心实现，可以参考《Linux内核完全剖析》、《Linux内核设计与实现》、《鸿蒙操作系统设计原理与架构》等





Xinu的分层设计


应用程序层
文件系统层
网络通信层
设备驱动和管理层
实时时钟管理层
进程间通信层
进程协调层
进程管理层
内存管理层
硬件层

并发处理

- 传统程序是串行的（sequential）：程序员假定计算机是逐条执行代码且任何时刻机器只能执行一条语句
- 并发处理（concurrent processing）意味着在同一时间可以进行多项任务
  - N个独立程序同时在N个处理器上执行
  - N个独立程序在少于N个处理器上执行
  - 如果只有一个核，并发计算是否可行（如未特别说明，本课堂所面对的都是只有1个核的情况）
  - 如果多项计算同时进行，如何防止程序之间发生干扰，如在给定时间内一个输入输出设备的控制权只为一个程序所拥有
- 多任务（multitasking）或多道编程（multiprogramming）
  - 操作系统在多个程序间切换可用的处理器，允许一个处理器仅用几毫秒执行一个程序，随后转而处理其他程序
  - 从人的角度：所有程序都在执行

分时和实时

- 支持多任务的系统可以分成两大类：分时和实时

- 分时（timesharing）
  - 给予所有计算任务以相同的优先级，并且允许计算任务在任何时间开始或终止
  - 允许动态创建一项计算任务，在面向人类用户的计算机上颇受欢迎：使用浏览器时运行电子邮件程序或是运行一个后台应用播放音乐
  - 最主要的特征：一项计算任务所获得的处理器时间与这个系统的负载呈反比例关系——如果有N项计算任务正在进行，那么每项计算任务大约会获得可用CPU周期的1/N
  - 随着越来越多计算任务的出现，每项计算任务的速率将会不可避免地降低

分时和实时

- 实时（real-time）
  - 要满足（苛刻的）性能约束，所以不会同等地对待所有计算任务
  - 为每项计算任务分配一个优先级，并且需要非常小心谨慎地调度处理器以确保每项计算任务满足所要求的执行计划
  - 最主要的特征：总是将处理器分配给最高优先级的任务，即使其他任务正在等待
  - 例：通过提高语音传输任务的优先级，手机中实时系统可以确保会话不被中断，即使此时用户运行了天气查看程序或游戏程序

并发程序的设计考量

- 编写串行程序时，程序员会想象一个处理器没有中断和干扰的情况下按部就班执行这个程序
- 编写并发程序时，则必须采取一种不同的思维：想象多项计算任务同时执行
- 操作系统内部代码是适应并发性最好的例子：在任意给定时刻，可能有多个进程在执行
  - 最简单情况：每个进程执行的应用程序代码不会被其他进程同时执行
  - 然而必须考虑：多个进程调用同一个操作系统函数或执行同一条指令
  - 甚至：**操作系统可能会在任意时间进行进程切换**
  - **在多任务系统中，对于一个要执行的计算任务，其相对执行速度无法保证**
- 程序员必须确保*无论执行什么操作系统代码或者以何种顺序执行，所有进程都能正确完成预期的功能。*

```
void main(void) {
    resume( create(sndA, 1024, 20, “process 1”, 0) );
    resume( create(sndB, 1024, 20, “process 2”, 0) );
}
void sndA(void) { while(1) { putc(CONSOLE, ‘A’); } }
void sndB(void) { while(1) { putc(CONSOLE, ‘B’); } }
```

- 以上示例代码展示了创建两个新进程的并发程序（基于Xinu）
  - 主程序不直接调用sndA和sndB，而是调用操作系统函数create和resume；
  - create创建一个准备执行但暂时挂起的新进程，并返回一个称为进程标识符（process identifier）或进程ID（process ID）的整数值；
  - resume启动（即解除挂起）给定的进程，允许该进程开始执行。

普通函数调用在被调用函数完成之前不会返回，而进程创建函数create和resume在启动一个新进程后立即返回，这将使已有的进程与新进程并发地执行。

- 在Xinu中，所有进程都是并发执行的
  - 除非程序员显式控制进程间的交互，否则一个给定进程是独立于其他进程而持续执行的。
  - 例子中，第一个新进程（process 1）执行函数sndA中的代码，不断发送字母A；
  - 第二个新进程（process 2）执行sndB，不断发送字母B；
  - 输出结果是许多A和B的混合；
  - 执行主程序的进程，在第二次调用resume之后将会退出，但是不会影响到新创建的进程（它们会继续不停地发送A和B）。

如果上述示例发生在基于优先级的实时系统中，两个新创建的进程拥有最高优先级，而执行主程序的进程只拥有普通优先级，会发生什么情况？

多个进程共享一段代码

- 下列代码所创建的两个新进程均执行函数sndch的代码，create的最后两个参数指明了create将传递一个参数给新进程以及该参数的值。
- 两个进程能够在互不影响的前提下并发地执行
  - 每个进程拥有一份自己的参数和局部变量副本
  - 一个进程产生一系列A，另一个进程产生一系列B

```
void main(void) {
    resume( create(sndch, 1024, 20, “send A”, 1, ‘A’) );
    resume( create(sndch, 1024, 20, “send B”, 1, ‘B’) );
}
void sndch(char ch) { while(1) { putc(CONSOLE, ch); } }
```

局部变量、函数参数以及函数调用栈的存储空间与执行函数的进程有关，而不是与该函数的代码相关。

即便一个进程与其他进程共享了相同的代码，操作系统也必须为每个进程分配额外的存储空间。因此可用的内存数限制了能够创建的进程数量。

进程退出与进程终止

- 前文主程序执行到代码末尾时，初始进程停止执行
  - 使用术语进程退出（process exit）来描述这种情形
  - 一旦一个进程退出了，它就从系统中消失了
  - 普通函数返回，是该函数的活动记录从运行栈中弹出；当进程的最后一条活动记录从栈中弹出时，该进程退出
- kill提供了一种终止（terminate）进程的机制，无需等待进程退出
  - kill是create的逆操作——接收一个进程ID作为参数，并立即将该进程移除
  - 可以在任意时刻、任意函数嵌套层将一个进程终止
  - 当进程终止时，它将立即停止执行，所有分配给该进程的局部变量均消失
  - 如果当前进程通过kill终止自身，则对kill的调用永不返回，因为调用它的进程已经退出了

共享内存、竞争条件和同步

- Xinu中，每个进程有它自己的局部变量、函数参数和调用函数副本，但所有进程共享一组全局变量。
- 操作系统可以选择在任意时刻从一个进程切换到另一个进程，所以可能存在潜在的竞争条件（race condition）。
- 下例代码的输出是不可预知的

```
int32  n = 0;
void main(void) {
    resume( create(consume, 1024, 20, “cons”, 0) );
    resume( create(produce, 1024, 20, “prod”, 0) );
}
void produce(void) { int32 i;
    for(i=1; i<=2000; i++) n++;
}
void consume(void) { int32 i;
    for(i=1; i<=2000; i++)
        printf(“The value of  is %d \n”, n);
}
```

期望输出n的值为0~1999

- 通过独立的进程进行数据的生产与消费是十分常见的
  - 程序员如何同步生产者和消费者进程，从而使消费者可以接收每个生产出来的数据值?
  - ➢ 生产者必须等待消费者访问了当前数据项后才能产生另一个数据项
  - ➢ 消费者必须等待生产者生成下一个数据项
- 使两个进程正确协作需要仔细设计一种**同步机制**，其至关重要的约束是

> 在一个并发编程系统中，进程不应该在等待其他进程时仍然占用处理器

  - 进程在等待另一个进程时仍然执行指令，可以认为它进入了忙等待（busy waiting）。
- Xinu提供了一个信号量（semaphore）抽象：
  - 一个信号量系统由一个整数值和一组进程构成；
  - wait递减信号量的值，当结果为负时，调用wait的进程被加入等待进程集合中(P操作）；
  - signal执行相反操作，将信号量递增并允许等待进程之一继续运行（V操作）。

```
int32  n = 0;
void main(void) {
    sid32  produced, consumed;
    consumed = semcreate(0);
    produced = semcreate(1);
    resume( create(cons2, 1024, 20, “cons”, 2, consumed, produced) );
    resume( create(prod2, 1024, 20, “prod”, 2, consumed, produced) );
}
void prod2(sid32 consumed, sid32 produced) {
    int32 i;
    for(i=1; i<=2000; i++) {
        wait(consumed); n++; signal(produced);
    }
}
void cons2(sid32 consumed, sid32 produced) {
    int32 i;
    for(i=1; i<=2000; i++) {
        wait(produced);
        printf(“The value of  is %d \n”, n);
        signal(consumed);
    }
}
```

Xinu中类型命名方法

- Xinu对数据类型进行了更为细致的重命名，既可以定义一个类型的存储空间，又能定义一个类型的用途
  - C语言本身的基本数据类型，在不同平台上存储空间可能不一样：long在32位系统上可能需要32位存储空间，而在64位系统上可能需要32位空间（Windows），也可能需要64位空间（Linux）
  - 普通C语言开发可以引用标准库头文件<stdint.h>中的int32、int64等来显式指明类型空间
  - Xinu中对基本数据类型名称进行了重定义
  - 与操作系统概念相对应的类型，由助记符和数字组成：
    - ➢ sid32：定义了信号量标识符并且存储空间为32位整数
    - ➢ qid16：定义了队列标识符并且存储空间为16位整数
  - 数据类型定义在头文件<kernel.h>中

类型	含义
byte	无符号8位值
bool8	8位bool型（0,1）
int16	有符号16位整型
uint16	无符号16位整型
int32	有符号32位整型
uint32	无符号32位整型

3 硬件与运行时环境概述

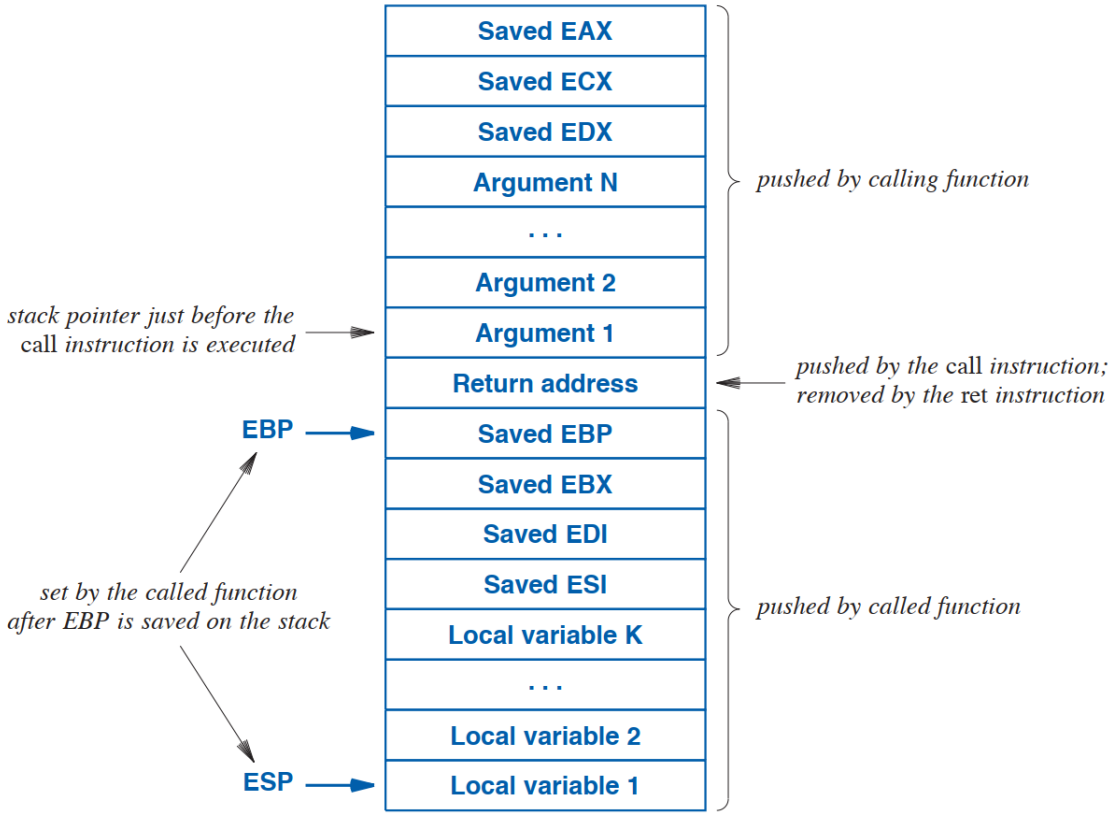
- 本课堂所讲述的内容以Xinu的Galileo版为主，针对Intel x86指令集（部分模块会介绍针对BeagleBone Black版的实现）
- 除了一些特殊情况，操作系统并不需要专注于指令集，因为大多数操作系统函数都是使用C语言编写的
  - 仅有非常少的汇编代码
- 32位Intel体系结构，包含8个32位通用寄存器
  - 每个寄存器的低16位寄存器可访问，如AX、BX
  - 部分寄存器的低16位又可分为两个8位寄存器，如AH和AL

名称	用途
EAX	累加器
EBX	基地址
ECX	计数
EDX	数据
ESI	源索引
EDI	目的索引
EBP	帧指针
ESP	栈指针

函数调用约定和运行时栈

- 操作系统严重依赖于“函数调用”这一机制
  - 应用程序通过函数调用引发系统服务，所以函数调用定义了系统服务
  - 当进行从一个进程到另一个进程的上下文切换时，操作系统必须处理进程各自正在进行一系列嵌套函数调用的情况
- 函数调用约定
  - 系统在实现函数调用与返回过程中采取的步骤被称为函数调用约定
  - 硬件不会指定所有细节；编译器设计者决定如何实现这一机制
- 参数和参数传递
  - 当一个函数被调用时，调用者会提供一个被称为实参的参数集合来匹配函数的形参
  - Intel 32位架构下，默认采用栈空间传递参数（64位平台下采用通用寄存器和栈空间共同传递参数）
- 运行时栈和栈帧内容
  - 使用运行时栈来存储与函数调用有关的状态，在栈中分配足够空间来保存被调用函数的活动记录，所分配的空间被称为栈桢
  - 栈桢包含了该函数的局部变量存储空间、计算所需临时存储空间、返回地址等

- 在Intel x86架构下，栈的生长方向为从较高的内存地址到较低的内存地址。
- 创建栈桢所需的代码由编译器生成——操作系统创建一个新进程时，要为这个进程初始化函数创建栈桢，所以操作系统需要确切知道栈桢的布局方式，包括参数与返回地址的位置等。
  - 参考前文例子中通过create向新进程传递的参数
- 下文以gcc编译器的函数调用约定为准



中断和中断处理

- 现代处理器提供了外部I/O设备在需要服务时中断处理器的机制
- 从操作系统角度来看，中断至关重要，因为它允许处理器在进行I/O的同时进行计算
- 硬件或操作系统必须保证：
  - 处理器的整个状态（包括程序计数器和状态寄存器）在发生中断时保存；
  - 处理器运行适当的中断处理程序，它在中断发生之前必须已经被放置在主存中；
  - 当中断完成时，操作系统和硬件提供恢复处理器整个状态并在中断点继续处理的机制。
- **中断可以在任何时候发生**，而操作系统也可以在中断期间从一个进程切换到另一个进程，这意味着其他进程随时都可能执行。

中断向量

- 设备中断时，硬件如何知道处理中断的代码的位置？
- 向量化中断的机制
  - 每个设备分配一个小的整数，如0、1、2等，称为*中断级别号或中断请求号*
  - 操作系统在内存中创建一个指针数组，称为**中断向量**
  - 中断向量数组中第i个条目指向处理具有向量号i的设备中断的代码
  - 中断发生时，设备通过总线将其向量号发送给处理器
  - 根据处理器细节，硬件或操作系统使用向量号作为中断向量的索引，获取指针，并使用指针作为运行代码的地址
- 中断向量必须在发生任何中断之前进行配置，因此操作系统必须在总线上分配设备地址的同时初始化中断向量。

时钟硬件

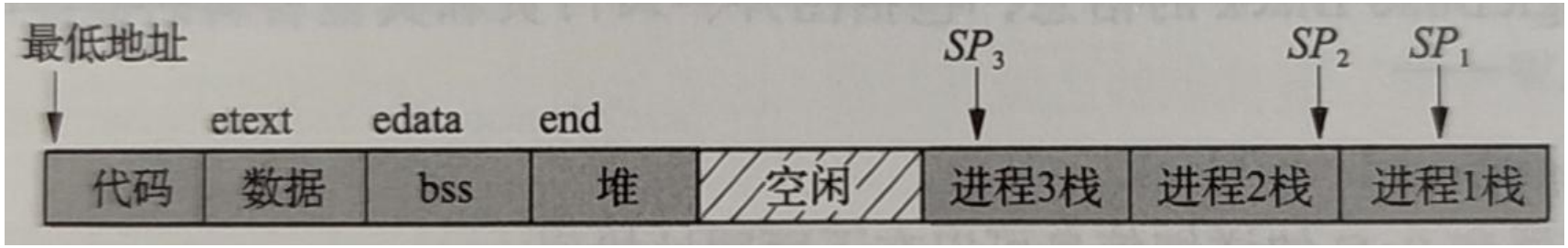
- 大多数计算机包含了可用于管理定时事件的硬件
- Xinu的x86版本主要针对实时时钟进行了处理
- 实时时钟
  - 电路由定期产生脉冲（例如，每秒1000次）的硬件组成
  - 要将实时时钟电路转化为实时时钟设备，需要把硬件配置为在每一个脉冲发生时都向处理器发送一个中断
  - 实时时钟设备不保留任何计数器，不存储时间，并且无法调整周期
- （详见后续的“实时时钟管理”章节）

串行通信

- 串行通信设备是最简单的I/O设备之一
- Xinu采用RS-232串行通信设备作为系统控制台
- 串行硬件既可以处理输入又可以处理输出（即字符的发送和接收）
  - Xinu使用RS-232将输入和输出统一处理，而无需专门处理键盘（输入）和显示器（输出）
  - QEMU中支持串行设备的模拟以及键盘/显示器的模拟
- 发生中断时，处理器检查设备硬件寄存器，以确定是输出完成传输还是输入端已经接收到字符
- （详见后续的“串口设备驱动”章节）

存储布局和内存保护

- 下图展示了Xinu多进程的内存分配机制
  - 所有进程共享代码段、数据段和bss段，但是所有进程必须分配独立的栈段



- 内存保护机制可以防止进程间的相互影响，也可保护操作系统免受应用程序进程的影响
  - 用户模式 v.s. 内核模式
- 与许多其他嵌入式系统一样，**Xinu代码完全以特权模式运行**，没有内存保护
  - 避免了内存保护的复杂性和运行时开销
  - 程序员必须格外小心，因为任何进程都可以访问任何内存位置

4 代码开发基础

- Xinu中仅包含少量的汇编代码
- 汇编代码以gcc能直接处理的AT&T格式存在
- 示例，针对如下C代码：
  - 在32位Linux平台下执行命令gcc –S test.c生成汇编代码文件test.s
  - 在64位Linux平台下执行命令gcc –S –m32 test.c生成汇编代码文件test.s
  - 以mov指令为例，基本格式为mov src, dst
  - mov可以使用b、w、l等后缀显式说明所操作数据的大小，第一个操作数为源操作数，第二个为目的操作数

```
// test.c
int add(int x, int y) {
    return x + y;
}
```

```
// test.s
add:
    pushl   %ebp
    movl    %esp, %ebp
    movl    8(%ebp), %edx
    movl    12(%ebp), %edx
    addl    %edx, %eax
    popl    %ebp
    ret
```

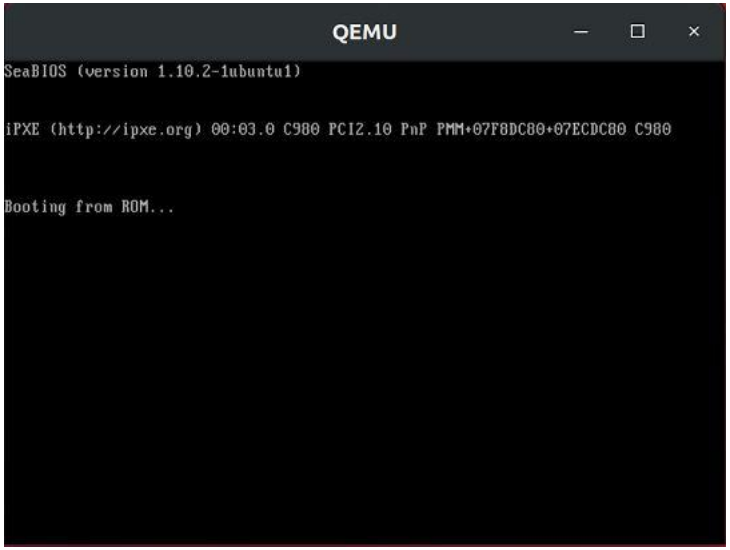
操作系统实现中，具体所需的汇编指令将在必要时进行说明

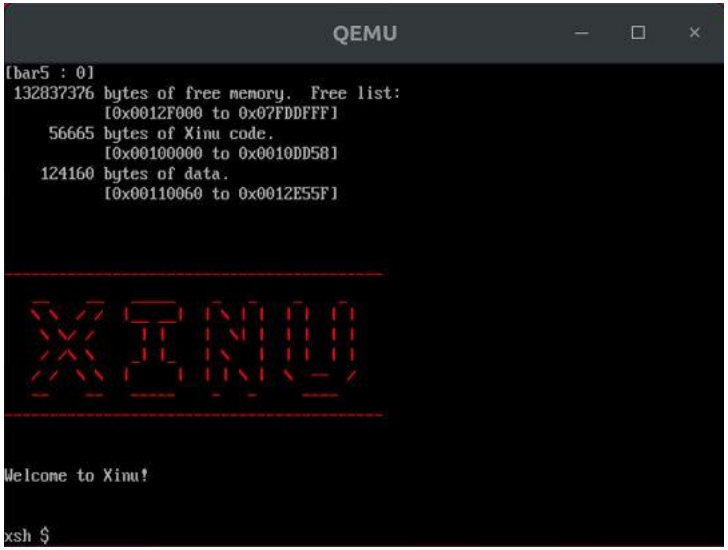
没有标准库函数可用

- 操作系统开发中没有任何可直接使用的库函数，如C开发中常见的printf、memset等
  - 在操作系统中要实现类似于printf的操作，需要首先完成驱动程序的编写（比如Xinu中针对RS-232串行设备），然后完成针对单个字符的输出，最后完成printf的功能
  - Xinu中实现了kprintf函数用于输出格式化的字符串

5 Xinu运行效果

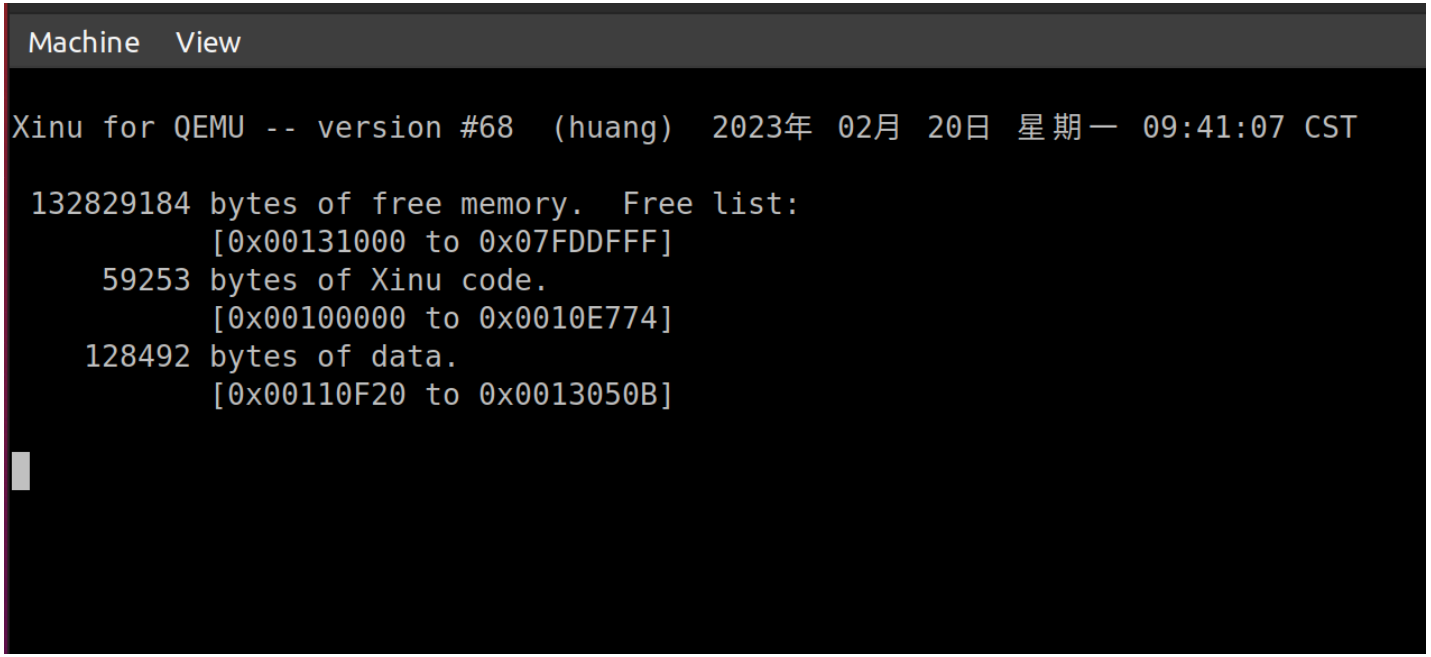
- 针对x86版本的Xinu代码，在QEMU中运行的效果如下图所示
  - Xinu本身不包含Bootloader
  - 假设代码编译后的二进制内核文件为xinu.elf
  - 执行命令：qemu-system-i386 –kernel xinu.elf，得到左图的界面
  - 按下Ctrl+Alt+3，进入Xinu启动完成后的shell界面





QEMU调试

- 以调试模式编译Xinu，生成xinu.elf
  - 打开两个终端窗口
  - 在窗口1执行：qemu-system-i386 -kernel xinu.elf -S -s
    - -S：在启动时就挂起CPU，等待gdb连接
    - -s：等价于-gdb tcp::1234，表示打开默认的1234这个gdb调试端口，也可以通过显示使用-gdb tcp::X的方式指定调试端口
  - 在窗口2执行：gdb ./xinu.elf
    - 连接上调试端口：target remote :1234
    - 设置断点：b main
    - 开始执行，QEMU中运行效果如下图所示



- 在gdb中继续执行，得到如下图的结果（进入Xinu的shell）
- gdb中的流程及命令如右图所示
- 如果想设置断点于某条具体指令处，应如何做？

```
Machine   View
 132829184 bytes of free memory.  Free list:
           [0x00131000 to 0x07FDDFFF]
     59253 bytes of Xinu code.
           [0x00100000 to 0x0010E774]
    128492 bytes of data.
           [0x00110F20 to 0x0013050B]

Welcome to Xinu!

xsh $
```

```
→ compile gdb ./xinu.elf
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04.1) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./xinu.elf...
(gdb) b main
Breakpoint 1 at 0x102423: file ../system/main.c, line 6.
(gdb) c
The program is not being run.
(gdb) target remote :1234
Remote debugging using :1234
0x0000fff0 in ?? ()
(gdb) c
Continuing.

Breakpoint 1, main () at ../system/main.c:6
6       {
(gdb) disassemble
Dump of assembler code for function main:
=> 0x00102423 <+0>:     endbr32
   0x00102427 <+4>:     lea    0x4(%esp),%ecx
   0x0010242b <+8>:     and    $0xfffffff0,%esp
   0x0010242e <+11>:    pushl  -0x4(%ecx)
   0x00102431 <+14>:    push   %ebp
   0x00102432 <+15>:    mov    %esp,%ebp
   0x00102434 <+17>:    push   %ebx
   0x00102435 <+18>:    push   %ecx
   0x00102436 <+19>:    call   0x10068e <__x86.get_pc_thunk.bx>
   0x0010243b <+24>:    add    $0xfbc5,%ebx
   0x00102441 <+30>:    call   0x10374a <recvclr>
   0x00102446 <+35>:    sub    $0x8,%esp
   0x00102449 <+38>:    push   $0x0
   0x0010244b <+40>:    push   $0x1
   0x0010244d <+42>:    lea    -0x4f64(%ebx),%eax
   0x00102453 <+48>:    push   %eax
   0x00102454 <+49>:    push   $0x32
   0x00102456 <+51>:    push   $0x2000
   0x0010245b <+56>:    mov    $0x10a3c1,%eax
   0x00102461 <+62>:    push   %eax
   0x00102462 <+63>:    call   0x100939 <create>
   0x00102467 <+68>:    add    $0x20,%esp
--Type <RET> for more, q to quit, c to continue without paging--q
Quit
(gdb) c
Continuing.
(gdb)
```

6 推荐阅读

- 《现代操作系统》（原书第4版），Andrew S. Tanenbaum，Herbert Bos 著，陈向群 译，第1章
- 《操作系统设计：Xinu方法》（原书第2版），[美] 道格拉斯·科默（Douglas Comer） 著，陈向群 郭立峰等 译，第1~3章
- 《鸿蒙操作系统设计原理与架构》，李毅，任革林 著，第1~3章